

EMSI CASABLANCA

Cycle Ingénieur — Data Science & Intelligence Artificielle

CAHIER DES CHARGES

Plateforme Intelligente de Détection de Fraude et

d'Analytique des Paiements ISO 20022

Graphes (Neo4j) · Machine Learning (XGBoost) · Tableau de bord (Streamlit)

Réalisé par :	Mohammed — Étudiant Ingénieur, 2ème année
Spécialité :	Data Science & Intelligence Artificielle
Type de projet :	Projet Fin d'Année (PFA) — Portfolio
Date :	Juillet 2026
Version :	1.0

Table des matières

1. Contexte du projet.....	3
2. Problématique.....	3
3. Objectifs généraux et spécifiques.....	4
4. Périmètre du projet.....	4
5. Analyse fonctionnelle.....	5
6. Exigences fonctionnelles.....	5
7. Exigences non fonctionnelles.....	6
8. Architecture générale.....	7
9. Architecture Neo4j.....	7
10. Pipeline Machine Learning.....	8
11. Description des modules Streamlit.....	8
12. Modèle de données.....	9
13. Planning prévisionnel.....	10
14. Livrables attendus.....	10
15. Perspectives d'amélioration.....	10

1. Contexte du projet

Les institutions financières traitent chaque jour des volumes massifs de messages de paiement au format ISO 20022 (pacs.008 pour les virements, pacs.002 pour les accusés de statut). Ces flux transitent entre banques émettrices et réceptrices à travers de multiples pays, devises et canaux de compensation.

Analyser ces flux avec des outils relationnels classiques atteint vite ses limites : les bases de données SQL décrivent bien des transactions isolées, mais expriment mal les relations entre comptes, banques et bénéficiaires — relations pourtant essentielles pour repérer des schémas de fraude (transferts circulaires, bénéficiaires partagés, réseaux de mules bancaires).

Ce projet propose de combiner trois approches complémentaires : une base de données orientée graphe (Neo4j) pour modéliser et explorer les relations entre entités, un modèle de Machine Learning (XGBoost) pour estimer un score de risque par transaction, et un tableau de bord interactif (Streamlit) pour rendre ces analyses accessibles à un utilisateur métier.

1.1 Origine et données disponibles

Le projet s'appuie sur deux sources de données de nature différente, ce qui constitue en soi une contrainte de conception importante à documenter :

- Un jeu de données synthétique de transactions ISO 20022 (fichiers iso20022_analytics.csv, pacs002.csv et pacs008.csv), comportant 5 000 transactions avec montants, devises, BIC émetteur/receveur, pays du débiteur et du créancier, code d'objet du paiement, statut de traitement et motif de rejet.
- Un export du schéma d'une base de données réelle de simulation de paiements (fichier dataa), décrivant 10 tables : campagnes de test, messages pacs.008/pacs.002, accusés de réception, simulations et leurs décisions (« GSIMT », « bénéficiaire »).

Ce second fichier ne contient pas de données transactionnelles mais uniquement la structure de la base (nom des tables, colonnes, types, nullabilité). Il servira de référence de modélisation — pour aligner le schéma Neo4j et les features du modèle sur une architecture réaliste de plateforme de test de paiements — plutôt que de source de lignes à charger directement.

2. Problématique

Comment concevoir une plateforme capable, à partir de messages de paiement ISO 20022, de (1) modéliser les relations entre comptes et banques sous forme de graphe pour révéler des réseaux suspects invisibles en vue tabulaire, (2) estimer un score de risque par transaction à l'aide d'un modèle supervisé, et (3) restituer ces résultats dans un outil interactif exploitable par un analyste ?

Constat sur les données

L'inspection du jeu de données synthétique montre un point structurant pour la suite du projet : aucune des trois sources ne contient d'étiquette explicite « fraude ». Le fichier d'analytique fournit en revanche un statut de traitement (TxSts) et, pour les transactions rejetées, un code de motif de rejet (RejectionReason) — 320 rejets sur 5 000 transactions, soit un taux de 6,4 %, répartis sur des codes tels que AC01 (compte invalide), AM04 (fonds insuffisants), BE04 (agent destinataire invalide) ou FF01 (format de fichier invalide).

Le schéma réel, de son côté, expose des champs de décision (decision, actual_gsimt_decision, actual_benef_decision) qui relèvent d'un contrôle de conformité / filtrage de sanctions plutôt que d'une

fraude au sens strict. Le projet doit donc être honnête sur ce point : il construira un score de risque / probabilité d'anomalie à partir de ces signaux de rejet et de décision utilisés comme cibles de substitution (proxy labels), et non un détecteur de fraude entraîné sur des cas confirmés et audités. Cette précision méthodologique est présentée en section 7.2.

3. Objectifs généraux et spécifiques

3.1 Objectif général

Concevoir et développer une plateforme d'analyse et de détection de risque sur des paiements ISO 20022, combinant base de données graphe, Machine Learning et tableau de bord interactif, avec un niveau de qualité suffisant pour constituer un projet de portfolio présentable en entretien technique.

3.2 Objectifs spécifiques

1. Importer, nettoyer et profiler les jeux de données ISO 20022 disponibles (pacs.008, pacs.002, vue analytique).
2. Concevoir un schéma de graphe Neo4j unifié (comptes, banques, transactions, pays) alimenté depuis les données transactionnelles.
3. Écrire des requêtes Cypher d'analyse de réseau (plus gros émetteurs/receveurs, transferts circulaires, bénéficiaires partagés, clusters connectés).
4. Construire un pipeline de feature engineering et entraîner un modèle XGBoost estimant une probabilité de rejet/anomalie par transaction.
5. Évaluer le modèle (précision, rappel, F1, ROC-AUC, matrice de confusion) et produire une explicabilité (importance des variables, SHAP).
6. Développer un tableau de bord Streamlit multi-pages : KPIs, exploration des données, analytique, prédiction, visualisation du graphe, évaluation du modèle, transactions suspectes.
7. Documenter l'ensemble (présent cahier des charges, README, présentation) pour un usage portfolio / entretien.

4. Périmètre du projet

4.1 Inclus dans le périmètre

- Traitement des trois fichiers CSV disponibles (analytique, pacs.002, pacs.008) comme source de données transactionnelles.
- Modélisation Neo4j et requêtes d'analyse de graphe sur ces mêmes transactions.
- Modèle XGBoost supervisé, entraîné sur cible de substitution (rejet / statut) faute d'étiquette de fraude confirmée.
- Application Streamlit locale, mono-utilisateur, connectée à Neo4j et au modèle entraîné.

4.2 Hors périmètre (phase 1)

- Ingestion temps réel de flux de paiement (traité en perspective, section 12).
- Chargement direct des tables réelles du fichier de schéma (dataa) : seule leur structure est utilisée comme référence de modélisation, aucune donnée réelle n'étant fournie.

- Authentification multi-utilisateurs et gestion fine des rôles.
- Déploiement en production (cloud, CI/CD) — un déploiement local/Docker est prévu, un déploiement cloud reste une perspective.

5. Analyse fonctionnelle

5.1 Acteurs

Acteur	Rôle vis-à-vis du système
Analyste risque / conformité	Explore les transactions, consulte les scores de risque, examine le graphe des réseaux suspects
Data Scientist (le porteur du projet)	Entraîne, évalue et met à jour le modèle ; maintient le pipeline de données
Administrateur technique	Gère l'import des données, la base Neo4j et le déploiement de l'application

5.2 Cas d'utilisation principaux

- Importer un nouveau fichier de transactions et lancer le pipeline de nettoyage.
- Consulter les indicateurs clés (volume, montant total, taux de rejet, répartition par pays/devise).
- Rechercher une transaction, un compte ou une banque et visualiser ses connexions dans le graphe.
- Soumettre les caractéristiques d'une transaction et obtenir une probabilité de risque en temps réel.
- Consulter la liste des transactions les plus à risque, triées par score, et exporter un extrait.
- Consulter les métriques de performance du modèle et l'importance des variables.

6. Exigences fonctionnelles

6.1 Module 1 — Import et préparation des données

- Import des fichiers CSV (analytique, pacs.002, pacs.008) avec validation des colonnes attendues.
- Détection et traitement des valeurs manquantes (le champ RejectionReason est nul par construction pour les transactions acceptées — il doit être encodé comme catégorie « aucun rejet », non comme donnée manquante à imputer).
- Détection des doublons (Messageld / EndToEndId) et rapport de profilage (types, cardinalités, distributions).

6.2 Module 2 — Analytique descriptive

- Nombre de transactions, montant total et moyen, par devise et par période.
- Répartition par pays émetteur/receveur, par banque (BIC) et par code d'objet de paiement (PurposeCode).
- Taux de rejet global et par code de motif de rejet.

6.3 Module 3 — Graphe Neo4j

- Création des noeuds Compte, Banque, Transaction, Pays et Devise à partir des BIC et des identifiants de message.
- Création des relations d'émission, de réception, de rattachement bancaire et de localisation (détail en section 8).
- Requêtes Cypher : plus gros émetteurs/receveurs, transferts circulaires ($A \rightarrow B \rightarrow C \rightarrow A$), comptes partageant un même bénéficiaire, sous-graphes fortement connectés autour d'une transaction à risque.

6.4 Module 4 — Machine Learning

- Prétraitement : encodage des variables catégorielles (devise, pays, banque, code d'objet), mise à l'échelle des montants, encodage cyclique de l'heure/jour.
- Feature engineering : fréquence de transaction par compte, historique de rejets par BIC émetteur, écart au montant moyen du couple (pays émetteur, pays receveur).
- Entraînement d'un modèle XGBoost sur la cible de substitution (rejet / statut), avec séparation train/test stratifiée.
- Évaluation par accuracy, précision, rappel, F1-score, courbe ROC/AUC et matrice de confusion, avec attention particulière au déséquilibre des classes (6,4 % de rejets).

6.5 Module 5 — Tableau de bord Streamlit

- Page d'accueil avec indicateurs clés.
- Explorateur de données avec recherche, filtres, pagination et export CSV.
- Page analytique avec graphiques interactifs (Plotly).
- Page de prédiction : saisie manuelle d'une transaction, restitution d'une probabilité de risque et d'un niveau (faible/moyen/élevé).
- Visualisation interactive du graphe Neo4j avec recherche d'une entité et affichage de son voisinage.
- Page d'évaluation du modèle (courbes, importance des variables, éventuellement SHAP).
- Page des transactions suspectes, triée par score de risque décroissant.

7. Exigences non fonctionnelles

Catégorie	Exigence
Performance	Chargement du tableau de bord en moins de 3 secondes sur 5 000 transactions ; prédiction unitaire en moins de 1 seconde
Maintenabilité	Code organisé en modules Python indépendants (préparation, graphe, ML, dashboard) avec README par module
Portabilité	Exécution locale via un environnement virtuel Python et Docker Compose (application + Neo4j)
Reproductibilité	Graines aléatoires fixées ; versions des dépendances figées dans requirements.txt
Traçabilité	Journalisation des étapes du pipeline (nombre de lignes en entrée/sortie de chaque étape)
Rigueur méthodologique	Toute limite de données (absence d'étiquette de fraude confirmée) explicitement documentée et communiquée dans le dashboard

8. Architecture générale

Le projet est organisé en modules faiblement couplés, chacun responsable d'une étape du pipeline (données brutes → graphe / features → modèle → restitution). Cette séparation facilite les tests unitaires et l'évolution indépendante de chaque composant.

```
FraudDetectionProject/
├── data/
│   ├── raw/           # CSV sources (analytics, pacs002, pacs008)
│   ├── processed/     # Données nettoyées, features
│   └── models/        # Modèle XGBoost sérialisé (.pkl)
├── neo4j/
│   ├── import_graph.py # Construction des noeuds et relations
│   └── queries.py      # Requêtes Cypher réutilisables
├── preprocessing/
│   ├── cleaning.py
│   └── feature_engineering.py
├── ml/
│   ├── train.py
│   ├── predict.py
│   └── evaluate.py
├── dashboard/
│   └── pages/          # Accueil, Explorateur, Analytique, Prédiction, Graphe,
├── Evaluation
│   └── components/
├── utils/
├── app.py
├── docker-compose.yml
└── requirements.txt
```

9. Architecture Neo4j

9.1 Noeuds

Noeud	Propriétés principales	Source
Compte	bic, code_banque	DbtrAgtBIC / CdtrAgtBIC
Banque	bic, pays	DbtrAgtBIC/CdtrAgtBIC + pays associé
Transaction	message_id, end_to_end_id, montant, devise, statut, motif_rejet, horodatage	iso20022_analytics.csv
Pays	code_iso	DbtrCountry / CdtrCountry

9.2 Relations

- (Compte)-[:ENVOIE]->(Transaction) — le compte émetteur d'une transaction.
- (Transaction)-[:RECUE_PAR]->(Compte) — le compte destinataire.
- (Compte)-[:RATTACHE_A]->(Banque) — banque teneuse du compte.
- (Banque)-[:LOCALISEE_EN]->(Pays) — pays de la banque.

9.3 Exemples de requêtes Cypher cibles

- Identifier les 10 comptes émetteurs cumulant les plus gros montants sur la période.
- Détecter les cycles de transferts (A→B→C→A) sur une fenêtre temporelle donnée.

- Repérer les comptes receveurs partagés par un grand nombre d'émetteurs distincts en peu de temps.
- Extraire le sous-graphe des transactions rejetées et de leurs comptes connexes, pour visualisation.

10. Pipeline Machine Learning

10.1 Variables d'entrée envisagées

Variable	Origine	Traitement
Montant (InstdAmt)	Colonne source	Normalisation + log-transformation
Devise (Currency)	Colonne source	One-hot encoding (3 modalités)
Pays émetteur / receveur	DbtrCountry / CdtrCountry	One-hot ou target encoding
Banque émettrice / réceptrice	DbtrAgtBIC / CdtrAgtBIC	Fréquence historique de rejet par BIC
Code d'objet du paiement	PurposeCode	One-hot encoding (9 modalités)
Heure / jour de la transaction	TxDateTime	Encodage cyclique (sin/cos)
Temps de traitement	ProcessingTimeSecs	Utilisé tel quel / discrétisé
Fréquence récente du compte	Calculée	Nombre de transactions du même émetteur sur 24h/7j

10.2 Cible du modèle

En l'absence d'étiquette de fraude confirmée (cf. section 2), la cible retenue est binaire : transaction rejetée (TxSts = RJCT) versus acceptée (TxSts = ACSP). Le tableau de bord présente le résultat comme une « probabilité de risque / anomalie », formulation volontairement distincte d'une « probabilité de fraude », pour rester fidèle à ce que les données permettent réellement d'apprendre.

10.3 Entraînement et évaluation

Séparation stratifiée train/test (respectant la proportion de 6,4 % de rejets), validation croisée pour le réglage des hyperparamètres XGBoost (profondeur, taux d'apprentissage, nombre d'arbres), et pondération des classes (scale_pos_weight) pour compenser le déséquilibre.

- Métriques : exactitude, précision, rappel, F1-score.
- Courbe ROC et aire sous la courbe (AUC).
- Matrice de confusion.
- Importance des variables (gain XGBoost) et, en extension, valeurs SHAP pour l'explicabilité par transaction.

11. Description des modules Streamlit

11.1 Accueil

- KPIs : nombre total de transactions, montant total, taux de rejet, score de risque moyen.

11.2 Explorateur de données

- Recherche par identifiant, filtres (pays, devise, statut, plage de montant), pagination, export CSV.

11.3 Analytique

- Évolution temporelle des paiements, distribution des montants, top banques et pays, usage des devises, heatmap horaire.

11.4 Prédiction de risque

- Formulaire de saisie d'une transaction → probabilité de risque → niveau (faible/moyen/élevé).

11.5 Visualisation du graphe Neo4j

- Recherche d'une entité (compte, banque, transaction) et affichage interactif de son voisinage direct.

11.6 Évaluation du modèle

- Courbe ROC, précision/rappel, matrice de confusion, importance des variables.

11.7 Transactions suspectes

- Table triée par score de risque décroissant, avec mise en évidence des transferts circulaires détectés côté graphe.

12. Modèle de données

Le schéma ci-dessous synthétise les tables du système réel de référence (fichier dataa), utilisé pour aligner la modélisation Neo4j et les futures features sur une architecture de plateforme de test de paiements réaliste. Ces tables ne contiennent pas de lignes exploitables dans ce projet — seule leur structure guide la conception.

Table	Rôle métier déduit du schéma
campaign	Campagne de simulation regroupant plusieurs messages, avec statistiques d'exécution
campaign_message	Message individuel d'une campagne, avec décisions attendues/obtenues et montants agrégés
message / message_ack	Enveloppe technique d'un message ISO 20022 et son accusé de réception
message008_transactions	Détail d'une transaction pacs.008 (débitteur, créancier, comptes, agents)
message002_transactions	Détail d'un statut pacs.002 (motif, référence à la transaction d'origine)
message_pacs002 / message_pacs008	En-tête de groupe des messages (somme de contrôle, nombre de transactions)
simulation / simulation_message	Simulation de bout en bout avec décision finale (acceptée/rejetée, par filtre de conformité)

13. Planning prévisionnel

Semaine	Livrable
S1	Cahier des charges finalisé, mise en place de l'environnement et du dépôt
S2	Import, nettoyage et profiling des données
S3-S4	Modélisation et alimentation du graphe Neo4j, premières requêtes Cypher
S5-S6	Feature engineering et entraînement du modèle XGBoost
S7	Évaluation, explicabilité (SHAP) et itération sur les features
S8-S9	Développement du tableau de bord Streamlit (pages 1 à 4)
S10	Développement des pages restantes (graphe, évaluation, transactions suspectes)
S11	Tests, corrections, documentation (README, présentation)
S12	Finalisation, packaging Docker, revue finale

14. Livrables attendus

- Application Streamlit complète et fonctionnelle.
- Base de données Neo4j peuplée avec le script d'import associé.
- Modèle XGBoost entraîné et sérialisé, avec rapport d'évaluation.
- Code source structuré et versionné (dépôt Git).
- Documentation (présent cahier des charges, README technique).
- Support de présentation du projet.

15. Perspectives d'amélioration

- Ingestion en flux continu des messages via Kafka.
- Traitement à grande échelle avec Spark pour des volumes de production.
- Graph Neural Networks pour exploiter directement la structure du graphe dans la prédiction.
- Comparaison avec des approches non supervisées (Isolation Forest, autoencodeurs) pour la détection d'anomalies sans étiquette.
- Explicabilité étendue (SHAP/LIME) intégrée nativement au tableau de bord.
- Exposition d'une API REST (FastAPI) pour l'intégration avec d'autres systèmes.
- Conteneurisation complète et déploiement cloud, gestion des rôles utilisateurs.